

hyper2kvm Troubleshooting Guide

Comprehensive troubleshooting guide for common issues and their solutions.

Table of Contents

1. [Installation Issues](#)
 2. [Connection Problems](#)
 3. [Conversion Errors](#)
 4. [Performance Issues](#)
 5. [Driver Injection Failures](#)
 6. [Boot Problems](#)
 7. [Network Configuration](#)
 8. [Debugging Tools](#)
-

Installation Issues

Problem: Missing Dependencies

Symptom:

```
ModuleNotFoundError: No module named 'guestfs'
```

Solution:

```
# RHEL/Fedora
sudo dnf install qemu-img qemu-system-x86

# Ubuntu/Debian
sudo apt-get install qemu-utils

# Verify installation
python3 -c "import guestfs; print('OK')"
```

Problem: Permission Denied

Symptom:

```
PermissionError: [Errno 13] Permission denied: '/dev/kvm'
```

Solution:

```
# Add user to necessary groups
sudo usermod -aG kvm $USER
sudo usermod -aG libvirt $USER

# Log out and log back in
# Verify
groups | grep -E "(kvm|libvirt)"
```

Problem: qemu-img Not Found

Symptom:

```
FileNotFoundError: qemu-img: command not found
```

Solution:

```
# Install QEMU tools
sudo dnf install qemu-img # RHEL/Fedora
sudo apt-get install qemu-utils # Ubuntu/Debian

# Verify
qemu-img --version
```

Connection Problems

Problem: vSphere Connection Timeout

Symptom:

```
VSphereConnectionError: Connection to vcenter.example.com timed out
```

Diagnosis:

```
# Test connectivity
ping vcenter.example.com

# Test HTTPS access
curl -k https://vcenter.example.com/sdk

# Check firewall
sudo iptables -L -n | grep 443
```

Solution:

```
# Add firewall rule
sudo firewall-cmd --permanent --add-rich-rule='
    rule family="ipv4"
    source address="vcenter.example.com"
    port port="443" protocol="tcp" accept'
sudo firewall-cmd --reload

# If behind proxy, set environment variables
export HTTPS_PROXY=proxy.example.com:8080
export NO_PROXY=localhost,127.0.0.1
```

Problem: SSL Certificate Verification Failed

Symptom:

```
SSLERROR: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed
```

Solution:

Option 1: Add CA certificate (Recommended)

```
# Get vCenter CA certificate
openssl s_client -connect vcenter.example.com:443 -showcerts < /dev/null 2>/dev/
null | \
    openssl x509 -outform PEM > /tmp/vcenter-ca.pem

# Add to system trust store
```

```
sudo cp /tmp/vcenter-ca.pem /etc/pki/ca-trust/source/anchors/  
sudo update-ca-trust  
  
# Verify  
openssl verify /etc/pki/ca-trust/source/anchors/vcenter-ca.pem
```

Option 2: Disable verification (Testing only)

```
# config.yaml  
vsphere:  
  verify_ssl: false # Use only for testing!
```

Problem: Authentication Failed

Symptom:

```
InvalidLogin: Cannot complete login due to an incorrect user name or password
```

Diagnosis:

```
# Test credentials manually  
govc about -u 'username@vsphere.local:password@vcenter.example.com'  
  
# Check account status in vCenter  
# GUI: Administration > Users and Groups
```

Solution:

```
# Reset password in vCenter  
# Update credentials in config  
  
# Use environment variables instead of config file  
export VSPHERE_USERNAME='administrator@vsphere.local'  
export VSPHERE_PASSWORD='correct-password'
```

Conversion Errors

Problem: VMDK Conversion Fails

Symptom:

```
ConversionError: qemu-img convert failed with exit code 1
```

Diagnosis:

```
# Test qemu-img manually
qemu-img convert -f vmdk -O qcow2 input.vmdk output.qcow2 -p

# Check disk space
df -h /var/lib/libvirt/images

# Verify VMDK integrity
qemu-img check input.vmdk
```

Solution:

If corrupted:

```
# Try to repair
qemu-img check -r all input.vmdk

# Or convert with error tolerance
qemu-img convert -f vmdk -O qcow2 -o compat=1.1 input.vmdk output.qcow2
```

If disk full:

```
# Free up space
sudo du -sh /var/lib/libvirt/images/* | sort -h
sudo rm -f /var/lib/libvirt/images/*.old

# Or use different output location
h2kvmctl local --vmdk input.vmdk --to-output /mnt/storage/output.qcow2
```

Problem: Image Inspection Fails

Symptom:

```
ImageInspectionError: guestfs_launch failed
```

Diagnosis:

```
# Check if KVM is available
ls -l /dev/kvm

# Test VMCraft backend
sudo modprobe nbd
ls /dev/nbd0

# Check for kernel panic in dmesg
dmesg | tail -50
```

Solution:

```
# If /dev/kvm missing, enable KVM
sudo modprobe kvm
sudo modprobe kvm_intel # or kvm_amd

# If still failing, use TCG (slower)
export LIBGUESTFS_BACKEND=direct
export LIBGUESTFS_HV=/usr/bin/qemu-system-x86_64

# Retry
h2kvmctl local --vmdk input.vmdk --to-output output.qcow2
```

Problem: Out of Memory

Symptom:

```
MemoryError: Cannot allocate memory
```

Diagnosis:

```
# Check available memory
free -h

# Check swap
swapon --show
```

```
# Monitor memory during conversion
watch -n 1 free -h
```

Solution:

```
# Add swap if needed
sudo dd if=/dev/zero of=/swapfile bs=1G count=8
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Limit guestfs backend memory
export LIBGUESTFS_MEMSIZE=2048 # MB

# Process images sequentially instead of parallel
h2kvmctl --config job.yaml # process images one at a time
```

Performance Issues

Problem: Slow Conversion Speed

Diagnosis:

```
# Monitor I/O
iotop -o

# Check disk performance
dd if=/dev/zero of=/var/lib/libvirt/images/test bs=1G count=1 oflag=direct

# Monitor CPU
top
```

Solution:

Use compression:

```
h2kvmctl local --vmdk input.vmdk --to-output output.qcow2 --compress
```

Increase parallelism:

```
h2kvmctl --config batch.yaml # with parallel_processing: true
```

Use faster storage:

```
# Use SSD/NVMe instead of HDD
h2kvmctl local --vmdk input.vmdk --to-output /mnt/nvme/output.qcow2
```

Tune qemu-img:

```
qemu-img convert \
  -f vmdk -O qcow2 \
  -o cluster_size=2M \
  -o preallocation=metadata \
  -m 4 \ # Use 4 coroutines
  input.vmdk output.qcow2
```

Problem: High Memory Usage

Diagnosis:

```
# Monitor memory
watch -n 1 'ps aux | grep hyper2kvm | head -5'

# Check for memory leaks
valgrind --leak-check=full python3 -m hyper2kvm ...
```

Solution:

```
# Limit memory
export LIBGUESTFS_MEMSIZE=1024

# Process smaller batches
h2kvmctl --config batch.yaml # reduce batch_size in config

# Close guestfs handles explicitly in code
# (Check for resource leaks)
```


Driver Injection Failures

Problem: virtio Drivers Not Found

Symptom:

```
DriverInjectionError: virtio_blk module not found in initramfs
```

Diagnosis:

```
# Check if VM boots without drivers
virt-inspector -a output.qcow2

# List modules in initramfs
guestfish -a output.qcow2 -i <<'EOF'
    cat /boot/initramfs-5.14.0.img | cpio -t | grep virtio
EOF
```

Solution:

Manual driver injection:

```
# Mount image
guestfish -a output.qcow2 -i

# Regenerate initramfs
><fs> command "/sbin/dracut -f"

# Or for Debian/Ubuntu
><fs> command "update-initramfs -u"

# Verify
><fs> cat /boot/initramfs-$(uname -r).img | cpio -t | grep virtio
```

Use auto-detection:

```
h2kvmctl local --vmdk input.vmdk --to-output output.qcow2 --regen-initramfs
```

Problem: GRUB Configuration Broken

Symptom:

```
grub rescue> Unknown command 'linux'
```

Diagnosis:

```
# Inspect GRUB config
guestfish -a output.qcow2 -i <<'EOF'
    cat /boot/grub2/grub.cfg
EOF

# Check for virtio disk references
grep virtio /boot/grub2/grub.cfg
```

Solution:

```
# Regenerate GRUB config
virt-customize -a output.qcow2 --run-command 'grub2-mkconfig -o /boot/grub2/
    grub.cfg'

# Or manually
guestfish -a output.qcow2 -i <<'EOF'
    command "/sbin/grub2-mkconfig -o /boot/grub2/grub.cfg"
EOF
```

Boot Problems

Warning: "initramfs rebuild failed: mtime+size unchanged"

Symptom:

```
initramfs rebuild failed: mtime+size unchanged (mtime=1769251671,
size=22324924)
```

This is NORMAL and expected for cloud-native distributions!

Explanation: - This warning means the initramfs **already contains virtio drivers** - Common with Photon OS, CoreOS, Flatcar, and other cloud-native distros - The rebuild detected drivers are present, so no changes were made - **The VM will boot successfully** - this is not an error

Verification:

```
# After conversion, check VM boots successfully
sudo virsh list --all           # VM should be running
sudo virsh domifaddr photon-converted # Should get IP address
nc -zv <IP> 22                  # SSH port should be accessible
```

When to investigate: - VM doesn't boot at all - Kernel panic about missing drivers -
Otherwise, ignore this warning - it's expected!

Related distributions that commonly show this: - VMware Photon OS (all versions) -
Fedora CoreOS - Flatcar Container Linux - RancherOS - K3OS

Problem: VM Won't Boot After Conversion

Diagnosis:

```
# Try booting with verbose output
virsh start vm-name --console

# Check libvirt logs
journalctl -u libvirtd -f

# Try rescue mode
virt-rescue -a output.qcow2
```

Solution:

Check disk configuration:

```
<!-- domain.xml -->
<disk type='file' device='disk'>
  <driver name='qemu' type='qcow2' cache='writeback' />
  <source file='/var/lib/libvirt/images/vm.qcow2' />
  <target dev='vda' bus='virtio' />
</disk>
```

Regenerate initramfs:

```
virt-customize -a output.qcow2 --run-command 'dracut -f'
```

Fix fstab:

```
virt-customize -a output.qcow2 --edit '/etc/fstab:s/sda/vda/g'
```

Problem: Libvirt Domain Doesn't Persist After Conversion

Symptom:

```
# After conversion completes successfully
sudo virsh list --all
# Domain is missing or shows as "shut off"

error: failed to get domain 'photon-converted'
```

Explanation: The `libvirt_test` smoke test creates **temporary domains** for testing that are automatically cleaned up after validation passes.

Solution:

Option 1: Use `keep_domain` configuration

```
# In your config file
libvirt_test: true
keep_domain: true      # Persist the domain after testing
vm_name: my-vm
```

Option 2: Manually define domain from XML

```
# Use pre-configured XML templates
sudo virsh define test-confs/photon-virtio.xml
sudo virsh start photon-converted

# Or create your own XML
sudo virsh define /path/to/domain.xml
sudo virsh start your-vm-name
```

Option 3: Skip smoke test and define manually

```
# In your config file
libvirt_test: false    # Skip automatic testing
```

Then define manually after conversion:

```
sudo virsh define your-domain.xml
sudo virsh start your-vm
```

Verify domain is persistent:

```
sudo virsh list --all --persistent
# Should show your domain with "yes" in persistent column
```

Problem: Kernel Panic on Boot

Symptom:

```
Kernel panic - not syncing: VFS: Unable to mount root fs
```

Diagnosis:

```
# Check fstab
guestfish -a output.qcow2 -i cat /etc/fstab

# Check initramfs modules
guestfish -a output.qcow2 -i cat /etc/dracut.conf
```

Solution:

```
# Add required modules
virt-customize -a output.qcow2 \
  --run-command 'echo "add_drivers+=\" virtio_blk virtio_scsi \"" >> /etc/
  dracut.conf.d/virtio.conf'

# Regenerate initramfs
virt-customize -a output.qcow2 --run-command
  'dracut -f --kver $(ls /lib/modules | head -1)'
```

Network Configuration

Problem: No Network After Boot

Diagnosis:

```
# Check if interface is detected
virsh domiflist vm-name

# Inside VM
ip link show
dmesg | grep virtio_net
```

Solution:

For NetworkManager:

```
virt-customize -a output.qcow2 --run-command 'nmcli con mod eth0
connection.interface-name ens3'
```

For systemd-networkd:

```
virt-customize -a output.qcow2 --write '/etc/systemd/network/20-wired.network:
[Match]
Name=en*

[Network]
DHCP=yes'
```

For static configuration:

```
virt-customize -a output.qcow2 --edit '/etc/sysconfig/network-scripts/ifcfg-
eth0:s/HWADDR=.*$/ '
```

Problem: Wrong Network Driver

Symptom:

```
No suitable device found: no device found for connection 'eth0'
```

Solution:

```
# Update network scripts to use virtio_net
virt-customize -a output.qcow2 --run-command '
    cd /etc/sysconfig/network-scripts
    for f in ifcfg-*; do
        sed -i "/HWADDR/d" "$f"
        sed -i "s/vmxnet3/virtio_net/g" "$f"
    done
'
```

Debugging Tools

Enable Debug Logging

```
# Environment variables
export LIBGUESTFS_DEBUG=1
export LIBGUESTFS_TRACE=1
export HYPER2KVM_DEBUG=1

# Run with verbose output
h2kvmctl local --vmdk input.vmdk --to-output output.qcow2 -v
```

Capture Detailed Logs

```
# Full debug log
hyper2kvm --debug --log-file /tmp/hyper2kvm-debug.log ...

# With strace
strace -f -o /tmp/strace.log hyper2kvm ...

# With ltrace
ltrace -o /tmp/ltrace.log hyper2kvm ...
```

Interactive Debugging

```
# Add breakpoint in code
import pdb; pdb.set_trace()
```

```
# Or use IPython
from IPython import embed; embed()
```

Test Individual Components

```
# Test image inspection only
virt-inspector -a input.vmdk

# Test conversion only
qemu-img convert -f vmdk -O qcow2 input.vmdk output.qcow2 -p

# Test driver injection only
virt-customize -a output.qcow2 --run-command 'dracut -f'
```

Getting Help

If you cannot resolve the issue:

1. **Check documentation:** <https://docs.hyper2kvm.io>
2. **Search GitHub issues:** <https://github.com/yourorg/hyper2kvm/issues>
3. **Ask on discussions:** <https://github.com/yourorg/hyper2kvm/discussions>
4. **Report a bug:** <https://github.com/yourorg/hyper2kvm/issues/new>

When reporting issues, include: - Operating system and version - hyper2kvm version
(`hyper2kvm --version`) - Full error message - Debug logs (`--debug --log-file debug.log`) - Steps to reproduce

Quick Reference

Common Commands

```
# Basic conversion
h2kvmctl local --vmdk vm.vmdk --to-output vm.qcow2

# With initramfs regeneration
h2kvmctl local --vmdk vm.vmdk --to-output vm.qcow2 --regen-initramfs
```



```
# Batch conversion (via config)
h2kvmctl --config batch.yaml # with parallel_processing: true

# From vSphere
h2kvmctl vsphere --vcenter vcenter.example.com --vm-name web-server-01 --to-output web-server-01.qcow2

# Dry run
h2kvmctl local --vmdk vm.vmdk --to-output vm.qcow2 --dry-run

# With manifest
h2kvmctl --manifest manifest.json --output-dir /var/lib/libvirt/images
```

Environment Variables

```
export LIBGUESTFS_DEBUG=1      # Enable guestfs backend debug
export LIBGUESTFS_TRACE=1      # Enable guestfs backend trace
export LIBGUESTFS_MEMSIZE=2048 # Set memory (MB)
export LIBGUESTFS_BACKEND=direct # Use direct backend
export HYPER2KVM_DEBUG=1       # Enable hyper2kvm debug
export VSPHERE_USERNAME=admin  # vSphere username
export VSPHERE_PASSWORD=pass  # vSphere password
```

See Also

- [Installation Guide](#)
- [Quick Start](#)
- [Security Best Practices](#)
- [API Reference](#)